EX 6 : BIDIRECTIONAL RNN VS FEEDFORWARD NN FOR TIME-SERIES PREDICTION

AIM:

To Implement a Bidirectional Recurrent Neural Network (BiRNN) to predict sequences in time-series data. Train the model and compare its performance with a traditional Feedforward Neural Network (FFNN) for sequence-based tasks.

OBJECTIVES:

1. Understand the application of RNNs and FFNNs for time-series forecasting.
2. Train a bidirectional RNN model to capture sequential dependencies.
3. Compare prediction performance with a feedforward neural network
4. Visualize and evaluate predictions using metrics such as Mean Squared Error (MSE).

ALGORITHM:

1. Import the required libraries, including Pandas, NumPy, PyTorch, Matplotlib, and scikit-learn.
2. Load the Airline Passenger dataset and convert the passenger counts into numerical values.
3. Normalize the data using MinMaxScaler to scale values between 0 and 1.
4. Create input sequences of 10 consecutive time steps and use the next value as the target.
5. Split the sequences into training and testing datasets in an 80:20 ratio.
6. Convert the datasets into PyTorch tensors and create DataLoaders for batch processing.
7. Define the Bidirectional RNN model with a recurrent layer and a fully connected output layer.
8. Define the Feedforward Neural Network with fully connected layers and ReLU activation.
9. Train both models using the Adam optimizer and Mean Squared Error (MSE) loss for 100 epochs.
10. Generate predictions on the test dataset using both trained models.
11. Apply inverse transformation to convert normalized predictions back to passenger counts.
12. Plot the actual values and predicted values of both models for comparison.
13. Calculate and compare the MSE of both models to evaluate their prediction performance.

SOURCE CODE:

```
# Step 1: Import Necessary Libraries
import pandas as pd
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
# ==========================================
# Step 2: Data Preparation
# ==========================================
url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/refs/heads/master/monthly-
airline-passengers.csv"
df = pd.read_csv(
   url,
   usecols=[1]
)
data = df.values.astype("float32")
# Normalize data between 0 and 1
scaler = MinMaxScaler(
   feature_range=(0, 1)
)
data_scaled = scaler.fit_transform(data)
# ==========================================
# Step 3: Create Sequences
# ==========================================
SEQ_LENGTH = 10
X = np.array([
   data_scaled[i:i + SEQ_LENGTH]
   for i in range(len(data_scaled) - SEQ_LENGTH)
]
```

```
y = np.array([
    data_scaled[i + SEQ_LENGTH]
    for i in range(len(data_scaled) - SEQ_LENGTH)
])
# Split into training and testing data
train_size = int(
    len(X) * 0.8
)
X_train = X[:train_size]
X_test = X[train_size:]
y_train = y[:train_size]
y_test = y[train_size:]


# ============================================
# Step 4: Prepare PyTorch Dataset and Loaders
# ============================================
X_train_tensor = torch.tensor(
    X_train,
    dtype=torch.float32
)
y_train_tensor = torch.tensor(
    y_train,
    dtype=torch.float32
)
X_test_tensor = torch.tensor(
    X_test,
    dtype=torch.float32
)
y_test_tensor = torch.tensor(
    y_test,
    dtype=torch.float32
```

```
)
train_loader = DataLoader(
    TensorDataset(
        X_train_tensor,
        y_train_tensor
    ),
    batch_size=16,
    shuffle=True
)
test_loader = DataLoader(
    TensorDataset(
        X_test_tensor,
        y_test_tensor
    ),
    batch_size=1,
    shuffle=False
)
# ============================================
# Step 5: Define BiRNN Model
# ============================================
class BiRNN(nn.Module):
    def __init__(
        self,
        input_size=1,
        hidden_size=64,
        num_layers=1
    ):
        super(BiRNN, self).__init__()
        self.rnn = nn.RNN(
            input_size,
            hidden_size,
```

```
            num_layers,
            batch_first=True,
            bidirectional=True
        )
        self.fc = nn.Linear(
            hidden_size * 2,
            1
        )

    def forward(self, x):
        out, _ = self.rnn(x)

        return self.fc(
            out[:, -1, :]
        )

# ================================================
# Feedforward Neural Network
# ================================================

class FeedforwardNN(nn.Module):
    def __init__(
        self,
        input_size
    ):
        super(FeedforwardNN, self).__init__()
        self.fc1 = nn.Linear(
            input_size,   64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(
            64,
```

```
            1
        )
    def forward(self, x):
        x = x.view(
            x.size(0),
            -1
        )
        x = self.relu(
            self.fc1(x)
        )
        return self.fc2(x)
# ============================================
# Step 6: Training Function
# ============================================
def train_model(
    model,
    loader,
    epochs=100
):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=0.01
    )
    model.train()
    for epoch in range(epochs):
        loss_epoch = 0.0
        for seqs, targets in loader:
            # Clear gradients
            optimizer.zero_grad()
            # Forward pass
```

```
            outputs = model(seqs)

            # Calculate loss
            loss = criterion(
                outputs,
                targets
            )
            # Backpropagation
            loss.backward()
            # Update weights
            optimizer.step()
            loss_epoch += loss.item()
        if (epoch + 1) % 20 == 0:
            print(
                f"Epoch {epoch + 1}/{epochs}, "
                f"Loss: "
                f"{loss_epoch / len(loader):.5f}"
            )
# ============================================
# Evaluation Function
# ============================================
def evaluate_model(
    model,
    X
):
    model.eval()
    with torch.no_grad():
        predictions = model(X).numpy()
    return predictions
# ============================================
# Step 7: Train Both Models
```

```
# ==========================================
print("Training BiRNN...")
birnn = BiRNN()
train_model(
    birnn,
    train_loader
)
print("\nTraining Feedforward Neural Network...")
ffnn = FeedforwardNN(
    input_size=SEQ_LENGTH
)
train_model(
    ffnn,
    train_loader
)
# Generate predictions
pred_birnn = evaluate_model(
    birnn,
    X_test_tensor
)
pred_ffnn = evaluate_model(
    ffnn,
    X_test_tensor
)
# ==========================================
# Step 8: Inverse Transform Predictions
# ==========================================
pred_birnn_inv = scaler.inverse_transform(
    pred_birnn
)
```

```
pred_ffnn_inv = scaler.inverse_transform(
    pred_ffnn
)

y_test_inv = scaler.inverse_transform(
    y_test
)

# ==========================================
# Plot Results
# ==========================================

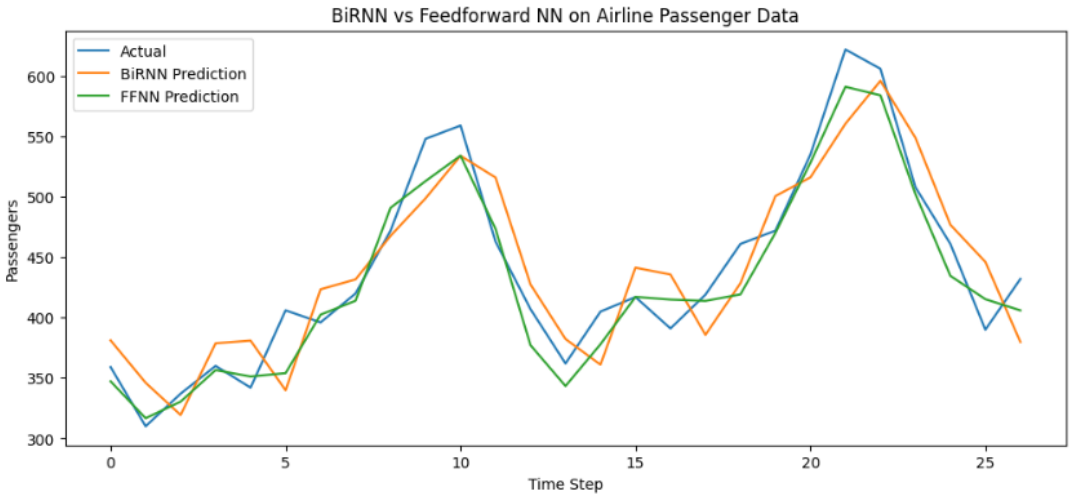
plt.figure(
    figsize=(12, 5)
)
plt.plot(
    y_test_inv,
    label="Actual"
)
plt.plot(
    pred_birnn_inv,
    label="BiRNN Prediction"
)
plt.plot(
    pred_ffnn_inv,
    label="FFNN Prediction"
)
plt.legend()
plt.title(
    "BiRNN vs Feedforward NN on Airline Passenger Data"
)
```

```
plt.xlabel(
    "Time Step"
)
plt.ylabel(
    "Passengers"
)
plt.show()
# ================================================
# Calculate MSE
# ================================================
mse_birnn = mean_squared_error(
    y_test_inv,
    pred_birnn_inv
)
mse_ffnn = mean_squared_error(
    y_test_inv,
    pred_ffnn_inv
)
print(
    f"BiRNN MSE: {mse_birnn:.3f}"
)
print(
    f"FFNN MSE: {mse_ffnn:.3f}"
)
```

OUTPUT:

```
Training BiRNN...
Epoch 20/100, Loss: 0.00257
Epoch 40/100, Loss: 0.00218
Epoch 60/100, Loss: 0.00253
Epoch 80/100, Loss: 0.00188
Epoch 100/100, Loss: 0.00251

Training Feedforward Neural Network...
Epoch 20/100, Loss: 0.00219
Epoch 40/100, Loss: 0.00175
Epoch 60/100, Loss: 0.00113
Epoch 80/100, Loss: 0.00074
Epoch 100/100, Loss: 0.00066
```




```
BiRNN MSE: 1314.362
FFNN MSE: 491.036
```

RESULT:

This experiment highlights the advantage of BiRNNs in modeling temporal dependencies in sequential data. While feedforward networks treat time steps independently, BiRNNs consider both past and future context, resulting in improved predictive performance. The MSE metric provides a quantitative basis for comparison.